

CS246 Spring 2026 – Constructor Project

Final Design Document (Due Date 2 Submission)

Team: Muneeb, Alex, Laith
Date Submitted: July 28, 2026

1. Introduction

This document describes the final design of Constructor, our implementation of a Settlers of Catan variant set at the University of Waterloo. It is intended to stand alone: no part of it assumes the reader has our source open. The final UML class model is submitted separately as `uml-final.pdf`, and a complete log of every divergence between that model and the original Due Date 1 model, with the specification section that forced each one, is maintained in the repository as `uml_changes.txt`.

2. Overview

The program is roughly 1800 lines of C++20 across 26 source files, organised with C++20 modules rather than headers. The module structure mirrors the design's layering:

- Five small standalone modules hold shared vocabulary types with no behaviour: `material`, `colour`, `residenceLevel`, `tileInfo`, and `subj_obs` (the Observer and Subject base classes). These sit at the bottom of the dependency graph and depend on nothing.
- Two standalone modules hold the pluggable strategies: `boardSource` (board setup) and `dice` (roll generation). Both are abstract bases with concrete subclasses, and neither knows anything about the board or the players.
- One module, `board`, holds the game model, split into six partitions: `:settlement`, `:vertice`, `:edge`, `:tile`, `:map`, and `:player`. These six types are mutually interdependent (a `Vertice` points at its `Edges`, an `Edge` points back at its `Vertices`), so they belong in one module; partitions let each still live in its own file.
- One module, `interface`, holds the controller: command parsing, the turn loop, command-line argument handling, and save/load.

The overall architecture is Model-View-Controller. The model is `Map` and the objects it owns (`Tile`, `Edge`, `Vertice`, `Settlement`) together with `Player`. The view is not a separate class: each model class implements a `display()` method that returns a `string` (or, for `Tile`, a small `TileInfo` struct) rather than printing anything itself, and `Map::display()` composes these into the full board. The controller is `Interface`, which is the only class in the program that touches `cin` or `cout` for game flow. This split is what allows the entire model layer to be reused unchanged under a different presentation, and it is discussed further under Resilience to Change.

3. Design

3.1 Ownership, RAII, and memory safety

Every owning relationship in the program is expressed through a vector or a `unique_ptr`; the program contains no `new` and no `delete` anywhere. `Interface` owns the `Map` through a `unique_ptr<Map>` and the four builders through a `vector<unique_ptr<Player>>`. `Map` owns its 19 `Tiles`, 72 `Edges`, and 54 `Vertices` through three vectors held by value. A `Vertex` exclusively owns the `Settlement` built on it through a `unique_ptr<Settlement>`. Every other pointer in the program is a raw pointer that deliberately expresses non-ownership: `Edge::road` and `Settlement::player` point at the builder who owns that piece, `Map::goosed` points at the tile currently holding the geese, and `Edge/Vertex` hold raw pointers to each other to represent board adjacency. Destruction is therefore entirely automatic and ordered correctly by construction: when `Interface` goes out of scope, the players and the map are released, and the map releases its tiles, edges, vertices, and their settlements in turn.

One consequence of this design is worth stating explicitly, because it drove a deliberate choice. Because `Settlement` and `Edge` hold raw `Player*` pointers into whatever container holds the players, that container must never reallocate after those pointers are taken. Storing players as `vector<Player>` by value would have made every one of those pointers dangle the first time the vector grew. Storing them as `vector<unique_ptr<Player>>` means the vector only ever relocates pointers, never the `Player` objects themselves, so the raw pointers stay valid for the life of the game. The same reasoning applies inside `Map`: its three vectors are built to their final size during construction, before any `Tile`, `Edge`, or `Vertex` takes a pointer into them, and are never resized afterward.

3.2 Board representation

The board is a graph, not a grid. `Map` holds three flat vectors and wires them together at construction: each `Edge` stores pointers to the two `Vertices` it joins, each `Vertex` stores pointers to the two or three `Edges` incident on it, and each `Tile` stores pointers to its six surrounding `Edges` and six surrounding `Vertices`. The specification notes that the board's shape is symmetric but not in a way that yields a simple formula, and suggests either hard-coding the relationships or reading them in. We generate them procedurally from a small set of row-pattern functions that describe which columns are occupied on each row of the board, which produces the vertex, edge, and tile numbering of Figure 3 exactly without a hard-coded table of 145 adjacency lists.

Because every adjacency question in the game reduces to a walk over this graph, the rules that depend on board geometry all live on `Map`, which is the only class that can see the whole structure: `canBuildRoad`, `canBuildResidence`, `buildersOnTile`, and the longest-road search added as an enhancement.

3.3 Observer pattern: resource distribution

When a builder rolls a non-seven, every tile whose value matches must pay out to every residence on it. Rather than have `Map` reach into each tile and each vertex to work out who gets what, `Map` derives from `Subject` and each `Tile` derives from `Observer`. Every tile registers with the map at construction. A roll becomes a single `Map::notify(rollValue)` call, which invokes `Tile::update(rollValue)` on all 19 tiles; a tile whose value does not match, or which is a `Park`, or which currently holds the geese, simply does nothing. A tile that does match walks its own six vertices and calls `giveMat` on each, which asks the settlement there to pay its owner an amount determined by the residence level. `Map` never needs to know how a tile decides to pay out, and `Tile` never needs to

know what triggered the roll.

3.4 Strategy pattern, applied twice

Two decisions in the game are genuinely interchangeable at runtime, and both are implemented as Strategy.

BoardSource is an abstract class with a single method, `getTiles()`, returning a `vector<TileInfo>`. `FileBoardSource` parses a layout file; `RandomBoardSource` generates a layout with the resource and value distribution the specification requires. `Interface` picks the appropriate one based on the command line, hands it to `Map`'s constructor, and `Map` calls `getTiles()` without knowing which kind it received.

`Dice` is an abstract class with a single method, `roll()`. `LoadedDice` prompts the user and validates the input; `FairDice` sums two pseudo-random values. Each `Player` holds a `unique_ptr<Dice>`, and `Player::setDice` replaces it in response to the `load` and `fair` commands. `Player::rollDice()` is a single delegating call, and the command handling never branches on dice type.

3.5 Residences: a level field, not a hierarchy

A basement, house, and tower differ in cost, in building points, in how many resources they collect, and in how they display. That makes an inheritance hierarchy tempting. We deliberately used a single `Settlement` class with a `ResidenceLevel` field instead, because in C++ an object's dynamic type is fixed at construction. If the three levels were subclasses, improving a residence would mean destroying one object and constructing a different one in its place, which would invalidate the `unique_ptr<Settlement>` in the vertex and break the invariant that a residence keeps the same identity and owner across upgrades. With a level field, `Settlement::improve()` advances an enum and every level-dependent quantity is derived from it, so the three values can never disagree with each other.

3.6 Where the rules live: check-then-act

Legality and mutation are deliberately separated. `Map` answers questions that require seeing the whole board (is this vertex adjacent to an existing residence; does this builder have a road reaching this edge), while `Edge` and `Vertex` enforce the local invariants only they can see (is this edge already built; is this vertex already occupied; is this residence already a tower) and perform the actual mutation. `Player` sits between them and always checks before acting: `Player::buildRoad` asks `Map::canBuildRoad` first, and only then calls `Edge::build`. Nothing in the type system enforces this ordering, so it is a convention the three of us agreed on explicitly during design review rather than something the compiler guarantees.

3.7 Error handling

Every operation that can legitimately fail returns `bool` and leaves state untouched on failure. `Interface` checks the result and prints the message the specification requires, and the game continues. This is a deliberate choice discussed further under Question 7.

3.8 Coupling and cohesion

The dependency graph is close to acyclic. The vocabulary modules depend on nothing. `boardSource` and `dice` depend only on vocabulary modules, and in particular neither knows that `Map` or `Player` exist, which is what makes them substitutable. The `board` module depends on the vocabulary modules and on the two strategy modules. `interface` depends on everything below it, and nothing depends on `interface`. The one deliberate exception is that `Player` holds a reference to `Map` so that it can act on the board, which is why the model types are one module with partitions rather than several separate modules; C++20 forbids the circular imports that separating them would require.

On the cohesion side, each class has one reason to change. `Tile` knows about a tile's resource, value, and geese state, and nothing about builders. `Settlement` knows about one residence's level and owner, and nothing about the board. `Interface` knows about input, output, and turn order, and nothing about how a road decides whether it can be built. The clearest evidence for this is that the model layer contains no `cout` statements at all: `display()` methods return strings and the controller decides what to do with them.

The one place we knowingly accepted extra coupling is that `Interface` reconstructs the vertex number of a residence by scanning `Map`'s vertices, because `Settlement` holds no back-pointer to the vertex it sits on. Adding that pointer would remove the scan at the cost of another cross-reference to keep consistent; with 54 vertices the scan is irrelevant, so we chose the simpler object.

4. Resilience to Change

We designed for a specification change we could not predict. The following are concrete, and in each case we state what would actually have to be edited.

A new way of setting up the board. Adding a source (a network layout, a procedurally generated variant, a fixed tutorial board) means writing one new class deriving from `BoardSource` and one new branch in `Interface`'s argument parsing. `Map` does not change at all, because it only knows it is handed something it can call `getTiles()` on. This is the whole reason the Strategy pattern is there rather than an `if` statement inside `Map`'s constructor.

A new kind of dice. Identical shape: one new class deriving from `Dice`, one new command in `Interface`. `Player` is untouched, because it only holds a `unique_ptr<Dice>` and calls `roll()`.

A new resource type. Resources are a single `Material` enumeration, and the per-builder counts are reachable as an ordered vector through one accessor. Adding a seventh resource means adding an enum value, extending the fixed-size resource vector and the name-to-enum parser, and revising the cost tables. Crucially, no class outside `Player` and the cost tables branches on which resource it is holding: tiles, vertices, and settlements pass a `Material` through without inspecting it.

A new residence level. Because residences are one class with a level field rather than a hierarchy, adding a fourth level means adding an enum value and extending the three small level-to-value mappings inside `Settlement` (cost, building points, display character). No new class, no change to `Vertex`, and no change to how upgrades are stored or how ownership is tracked.

A new command. Commands are dispatched in one place, in `Interface`'s turn handlers, and every command resolves to calls on the model that already exist. The two enhancement commands described in section 6 were added this way and required no change to any model class.

A different display, or a graphical front end. This is the change our layering protects against most deliberately. No model class prints anything; `Tile::display()` returns structured data rather than text, specifically so that a caller is free to render it differently. A graphical or curses front end would be a new controller alongside `Interface`, consuming the same `display()` results, with the entire board module reused unchanged. This is the direction discussed in Question 3.

A change to the building rules. The adjacency rules live in exactly two methods on `Map`. Changing the distance rule between residences, or allowing roads to be built through an opponent's residence, means editing one method each, and nothing else in the program has an opinion on the matter, because nothing else performs adjacency reasoning.

Turning features on and off. Both enhancements are gated behind a single boolean held by `Interface` and are switchable at runtime rather than only at startup. The pattern generalises: a feature with a command of its own is gated by refusing to recognise the command, and a feature without one is gated by passing the flag into the calculation it affects.

5. Answers to Specification Questions

All seven questions were answered in our Due Date 1 plan of attack. Each answer below states the final position and, where it changed, exactly how.

Question 1: choosing between a random board and a file at runtime

Unchanged from Due Date 1, and now implemented as described. We use the Strategy pattern: `BoardSource` is abstract with a single `getTiles()` method, implemented by `FileBoardSource` and `RandomBoardSource`. `Interface` selects one based on the command-line arguments and passes it to `Map`'s constructor. `Map` never learns which it received. We predicted this in the plan of attack and built it exactly that way; the only detail worth adding is that the strategy object is owned transiently during setup and released once the board exists, since nothing needs it afterward.

Question 2: switching between loaded and fair dice at runtime

Unchanged in design; one implementation defect is worth recording. `Dice` is abstract with a single `roll()`, and each `Player` holds a `unique_ptr<Dice>` swapped by `setDice` in response to `load` and `fair`. During integration we found that `Dice::roll` had originally been declared non-virtual, which meant that calls through the `unique_ptr<Dice>` resolved to the base class and never reached `LoadedDice` or `FairDice` at all. The pattern was correct on paper and inert in practice. The fix was to declare `roll()` pure virtual and give `Dice` a virtual destructor. The lesson is that Strategy depends entirely on dynamic dispatch actually being enabled, and that a polymorphic base class without a virtual destructor is a defect regardless of whether it has yet caused a visible leak.

Question 3: supporting different game modes

Unchanged from Due Date 1; still not implemented, and we can now be more specific about what it would cost. Our answer remains Strategy applied at the view and controller layer. A hexagonal board is a different `BoardSource` combined with a different adjacency-generation step inside `Map`'s constructor, both of which are already isolated. A graphical display is a second controller: because no model class prints, and because `Tile::display()` already returns structured data rather than a

formatted string, a new front end would consume the same model unchanged. Making that concrete would mean promoting `Interface` to an abstract base with `CLIInterface` and `GraphicalInterface` beneath it. We chose not to, because the specification requires exactly one interface, and introducing an abstraction with a single implementation adds indirection without removing any duplication. Different player counts are the one item here we are less well positioned for: the number four is assumed in several places in `Interface`, and generalising it would mean parameterising those loops.

Question 4: guaranteeing a legal action sequence across player types

Unchanged from Due Date 1; still not implemented. Our answer remains the Template Method pattern with the Non-Virtual Interface idiom. `Interface` already encodes the legal sequence of a turn in two non-public methods, `beginningOfTurn` and `duringTurn`, which run in a fixed order set by `play()`. Introducing computer players would mean promoting that sequence into a non-virtual `takeTurn()` on an abstract `PlayerController`, with the individual decisions inside it (which vertex to build on, whether to accept a trade) as private virtual methods overridden by `HumanController` and `ComputerController`. Because `takeTurn()` would be non-virtual, no subclass could reorder or skip a step, which is the guarantee the question asks for. We did not implement this because the specification has only human players, so the abstraction would have exactly one implementation.

Question 5: computer players with different strategies

Unchanged from Due Date 1; still not implemented. This composes with Question 4 rather than replacing it. `ComputerController` would hold a `unique_ptr<AIStrategy>` whose methods correspond to the decision points the template method already calls out to, with `EasyStrategy`, `MediumStrategy`, and `HardStrategy` implementing them differently. Changing difficulty at runtime would be a single `unique_ptr` reassignment, exactly the mechanism `Player::setDice` already uses for dice. Nesting `Strategy` underneath Template Method this way keeps the sequence of a turn fixed while leaving the decisions inside it free to vary, and avoids the combinatorial subclass growth that would follow from encoding each difficulty as its own controller subclass.

Question 6: changing tile production during a game

Unchanged from Due Date 1; still not implemented. Our answer remains the Decorator pattern. A tile's production step would be wrapped by decorators implementing the same interface, for instance one that adds a second resource type to whatever the wrapped tile produces and one that reduces the quantity on each successive trigger, each delegating to the object it wraps. This avoids a dedicated subclass for every combination of modifiers a tile might accumulate over a game. Our current `Tile::update` is the natural wrapping point, since it is already the single place where a tile decides what to pay out. We did not implement it because tile production is fixed for the entire game in this specification, with no mechanism by which a builder could alter it.

Question 7: exceptions

This answer has changed from Due Date 1, and the change is worth stating plainly. In the plan of attack we said we would not use exceptions at all. The architectural half of that claim held: no game rule, and no failure a player can cause through legal play, is signalled by an exception. Illegal build locations, insufficient resources, unrecognised commands, and out-of-range dice input are all expected

outcomes of ordinary input, and the specification requires a specific message and continued execution for each, so every such operation returns `bool` and `Interface` translates the result into the required output.

The literal half of the claim did not hold. Two parsing helpers, one converting a command argument to an integer and one validating loaded-dice input, call `std::stoi`, which throws on input it cannot parse. Both wrap that call in a narrow `try/catch` that immediately converts the exception into a `false` return. So the program does contain exception handling, but only at the boundary where the standard library forced the choice, and the exception never propagates further than the function that produced it. Given the chance to revise this, the cleaner course would have been to use the `istream` extraction idiom from the course notes, which reports failure through a stream state rather than by throwing, and would have made the codebase genuinely exception-free as originally claimed.

The one place we did consider exceptions on their merits was file I/O when loading a save or a board layout. A missing or malformed file is arguably exceptional rather than routine, and the specification defines no message for it. We check `ifstream`'s failure state once at startup instead, and treat a failed load as a false return, on the grounds that introducing a custom exception type for a case the specification does not require us to handle gracefully would add machinery without adding safety.

6. Extra Credit Features

6.1 Memory management without explicit deallocation

The program contains no `new`, no `delete`, and no `malloc` or `free` anywhere. All ownership is expressed with `vector` and `unique_ptr` as described in section 3.1, and every remaining raw pointer is documented above as deliberately non-owning. We verified this dynamically rather than only by inspection: running a scripted session covering opening placement, a full turn cycle with resource distribution, and a save under `valgrind --leak-check=full` reports 483 allocations, 483 frees, zero bytes still reachable at exit, and zero errors.

6.2 Bank trading

A builder may trade with the bank at a fixed four-to-one ratio through a new command, `bank <give> <take>`, available during the same phase as ordinary trading. It reuses the existing material parser and the existing resource adjustment methods on `Player`, and required no new method on any model class. Insufficient resources produce the same message the specification already defines for that situation. We chose a flat ratio rather than ports, because the board model in this specification has no concept of a port and inventing one would have required adjacency data with no basis in the rules.

6.3 Longest road bonus

The builder holding the longest continuous chain of their own roads receives two additional building points, provided that chain is at least five roads long. The bonus transfers only when another builder strictly exceeds the current holder's length, so a tie leaves it where it is; this matches the original game and prevents the bonus from oscillating between builders.

Longest road for a builder is the longest simple path through that builder's own edges. The search is a depth-first backtrack run from every vertex, which is comfortably affordable on a board of 72 edges.

One rule deserves emphasis because it is easy to omit: a path may pass through a vertex that is empty or holds that same builder's own residence, but is blocked from continuing through a vertex holding another builder's residence. Omitting that check yields a plausible-looking but incorrect answer in ordinary board positions, so it is implemented rather than simplified away. The holder is recomputed after a road is built and after a save is loaded, not on every query, since building a road is the only event that can change it.

The state lives on `Map` rather than `Player`, because `Map` is the only class that can see every builder's roads, while a `Player` has no access to another `Player`'s state. `Player::getBuildingPoints` asks the map whether it currently holds the bonus.

6.4 Enabling and disabling enhancements

Both features sit behind a single switch. The guidelines require an enhancement to be selectable by a command-line flag on the same binary rather than by compiling two versions, and add that it would be better still if enhancements could be turned on and off while the program runs. We implemented the second form. A `-enablebonus` flag sets the initial state, and the commands `enable-bonus` and `disable-bonus` change it at any point during play. Those two commands are always recognised, whatever the current state, since gating them behind the flag they control would make the feature impossible to switch on. With the switch off, `bank` is rejected exactly as any unrecognised command would be, and building-point totals are computed exactly as they were before either feature existed.

7. Final Questions

Question 1: lessons about developing software in teams

The most valuable thing we did was fix the interfaces before writing the code behind them. Because the UML model settled the exact signature and ownership semantics of every class in advance, three people could write against each other's classes before those classes existed. Where we deviated from that model, the cost was immediate and visible, which leads to the second lesson.

A shared design artefact is only useful if it stays true. We kept a running log, `uml_changes.txt`, recording every divergence between the diagram and the code together with the specification section that forced it. This began as bookkeeping for this document and turned into the most useful process decision we made: it meant that a signature change made by one person to satisfy a rule was visible to the others as a written justification rather than as a surprise at integration time. Several of the entries, including the two flags that distinguish opening placement from ordinary building, were changes none of us anticipated at design time and would have caused conflicting assumptions had they gone unrecorded.

Third, integration failures cluster at the seams, not inside the components. Our worst defect was not a wrong algorithm; it was that loading a saved game silently restored no roads, because the routine that rebuilds them charged each road its resource cost while the builders' resources had not yet been restored, and the failed return value was not checked. Each half of that was locally correct. Only the combination was wrong, and it was invisible without a test that saved a game and reloaded it. The general lesson is that the boundary between two people's correct work is where the bugs live, and that ignoring a `bool` return is how they stay hidden.

Finally, keeping the build green from day one was worth more than any single technique. The plan called for stubbing every class from the UML with empty bodies before implementing anything, so the project compiled and ran from the first day and never stopped. This meant we always had something submittable, and it meant a compile failure always pointed at the change we had just made rather than at an accumulated pile.

Question 2: what we would do differently

We would write the scripted tests much earlier. For most of the project we tested by playing the game by hand, which is slow, inconsistent between runs, and effectively impossible to repeat after a change. Late in the project we moved to feeding recorded command sequences through standard input with a fixed random seed, which made a full session reproducible and made memory checking under Valgrind straightforward. Every significant defect we found late would have been caught earlier and more cheaply by a handful of those scripts, and the save-and-reload defect in particular would have been caught the first time such a script existed.

We would also treat exact output text as data rather than as literals scattered through the controller. Because the specification grades on precise wording, a single misplaced prefix in a prompt is a real defect, and we shipped one for a while: the trade prompt was printed with a word the specification does not use. Collecting the required strings in one place would have made them checkable against the specification in one pass rather than by reading the whole controller.

On design specifically, we would give `Settlement` a back-pointer to the vertex it occupies. Omitting it kept the object smaller, but the controller now recovers vertex numbers by scanning the board in two separate places, which is more code in a more awkward place than the field would have been. Finally, we would decide the ownership model for the player collection before writing anything that points into it; changing it afterwards was mechanical but touched more of the program than it should have.

8. Conclusion

Constructor is complete against the specification, plus three enhancements. The design choices we are most confident in are the ones that isolated change: the two Strategy hierarchies, the Observer relationship that keeps the map and its tiles ignorant of each other's internals, the decision to model residences as one class with a level rather than as a hierarchy, and the strict separation between classes that decide and the controller that prints. The choices we would revisit are documented above. Every divergence between the delivered code and our original class model is recorded in `uml_changes.txt` with the rule that caused it, and the final model is submitted as `uml-final.pdf`.